

This is CS50

Week 7

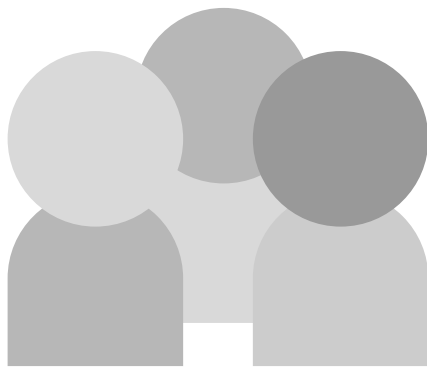
Kelly Ding (she/her)

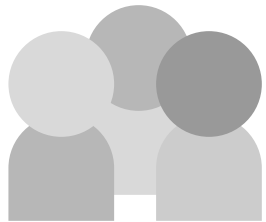
Preceptor

kelly@cs50.harvard.edu

Part 0

SQL Principles







Collection

Storage

Use

How can database design
choices impact the people
behind the data?

Quick Technical Design Principles

- Each table should be a collection of a **single entity**.
 - For example, we should have a different table for each of employees and *employee relationships*.

staff.db

employees

name	role
Alice	IT Staff
Laura	General Manager

employee_relationships

manager	employee
Laura	Alice

Quick Technical Design Principles

- Each table should be a collection of a **single entity**.
 - For example, we should have a different table for each of employees and *employee relationships*.
- Each piece of data should be stored in a **single location**, and thereafter referred to via its ID, aka primary key.
 - For example, we should ensure every employee an ID, and use those IDs in the *employee relationships* table.

staff.db

employees

id	name	role
1	Alice	IT Staff
2	Laura	General Manager

employee_relationships

manager_id	employee_id
2	1

Part 1

Database Design

Create a Database

In your terminal, use **sqlite3 friends.db** to create a new database, **friends.db**.

```
CREATE TABLE table_name (  
    column_name column_type specifiers  
);
```

Create a Table

Use **CREATE TABLE** to make a table named **friend**, with at least two columns, **id** and **name**, and at least one additional attribute you'd like to store. Denote **id** as the primary key.

```
CREATE TABLE "friend" (  
    "id",  
    "name",  
    "age",  
    PRIMARY KEY("id")  
);
```

```
CREATE TABLE "friend" (  
    "id" INTEGER,  
    "name" TEXT,  
    "age" INTEGER,  
    PRIMARY KEY("id")  
);
```



```
CREATE TABLE "friend" (  
    "id" INTEGER NOT NULL,  
    "name" TEXT NOT NULL,  
    "age" INTEGER NOT NULL,  
    PRIMARY KEY("id")  
);
```

```
INSERT INTO tablename (column1, column2)
VALUES (value1, value2);
```

Insert into a Table

Use **INSERT INTO** to add at least 3 friends to your table.

```
INSERT INTO friend (name, age)
VALUES ("Alice", 25);
```

```
UPDATE table_name  
SET column_name = new_value  
WHERE condition;
```

Update a Table

Suppose Alice has turned 26. Use **UPDATE** to change the age of Alice to 26.

```
UPDATE friend  
SET age = 26  
WHERE name = "Alice";
```

```
DELETE FROM table_name  
WHERE condition;
```


Delete a Row

Suppose you're no longer friends someone. Use **DELETE** to remove a friend from the **friend** table.

```
DELETE FROM friend  
WHERE name = "Bob";
```

```
SELECT * FROM tablename;
```

```
SELECT col_1, col_2 FROM table_name  
WHERE condition;
```

Songs

```
$ sqlite3 songs.db
```

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

Selecting

Ordering

Limiting

Aggregating

Selecting


```
SELECT name  
FROM songs  
WHERE duration < 240;
```

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

LIKE

% indicates wildcard characters in relative location of string:

WHERE name LIKE "%Love"	All names with "Love" at the end. Ex. <i>Somebody To Love</i>
WHERE name LIKE "Love%"	All names with "Love" at the beginning. Ex. <i>Love Yourself</i>
WHERE name LIKE "%Love%"	All names with "Love" somewhere in the name. Ex. <i>I Love Rock 'n' Roll</i>

1.sql

List the names of all songs in the database.

Ordering

```
SELECT *  
FROM songs  
WHERE tempo > 100  
ORDER BY tempo DESC;
```

```
SELECT *  
FROM songs  
WHERE tempo > 100  
ORDER BY tempo;
```

← ASC by default

2.sql

List the names of all songs in increasing order of tempo.

Limiting

```
SELECT *  
FROM songs  
WHERE tempo > 100  
ORDER BY tempo  
LIMIT 1;
```

Keyword Review

Keyword		Example	
*	Selects all columns.	<code>SELECT * FROM courses</code>	Returns every column of courses table.
LIKE " ____ "	Checks if a value is "like" the provided string. % serves as a placeholder for anything.	<code>SELECT code FROM courses WHERE code LIKE "CS%"</code>	Returns course codes that start with "CS". The % matches anything after "CS".
ORDER BY col_name ASC/DESC	Sorts by the col_name in either ascending or descending order. ASC by default.	<code>SELECT name FROM people ORDER BY name</code>	Returns names in alphabetical order
LIMIT n	Limits the number of results to n.	<code>SELECT name FROM people LIMIT 2</code>	Returns only the first two names in people .

3.sql

List the names of the top 5 longest songs,
in descending order of length.

4.sql

List the names of any songs that have *danceability*, *energy*, and *valence* greater than 0.75.

Aggregating

```
SELECT COUNT(*)  
FROM songs  
WHERE tempo > 100;
```

```
SELECT AVG(tempo)  
FROM songs;
```


Aggregation Keywords

AVG(col)	Returns the average of a column.
COUNT(col)	Returns the number of rows in the result.
DISTINCT(col)	Returns only distinct results.
query1 INTERSECT query2	Returns only results that are common to query1 and query2.

5.sql

Find the average energy of all the songs.

How might we reference
data across tables?

SELECTs

JOINs

SELECTs

JOINS

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

Query for all the songs by the
artist named "Oh Wonder"

What information do you need?

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic


```
SELECT *  
FROM songs  
WHERE artist_id =  
(  
    SELECT id  
    FROM artists  
    WHERE name = "Oh Wonder"  
);
```

```
SELECT *  
FROM songs  
WHERE artist_id =  
(  
    SELECT id  
    FROM artists  
    WHERE name = "Oh Wonder"  
);
```

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

```
SELECT *  
FROM songs  
WHERE artist_id =  
(  
    45  
);
```

```
SELECT *  
FROM songs  
WHERE artist_id = 45;
```

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

Query for all the songs by
artists born after 1980.

What information do you need?

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic


```
SELECT *  
FROM songs  
WHERE artist_id IN  
(  
    SELECT id  
    FROM artists  
    WHERE birthyear > 1980  
);
```

```
SELECT *  
FROM songs  
WHERE artist_id IN  
(  
    SELECT id  
    FROM artists  
    WHERE birthyear > 1980  
);
```

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

```
SELECT *  
FROM songs  
WHERE artist_id IN  
(  
    23,  
    45  
);
```

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

IN or =

We use **IN** instead of **=** when you're checking if something is **in** a list, rather than just equal to a single value.

SELECTs

JOINS

songs.db

songs

id	name	tempo	duration	artist_id
1	Something Comforting	144	282	23
2	Drive	142	196	45

artists

id	name	birthyear	label
23	Porter Robinson	1992	Mom+Pop
45	Oh Wonder	1990	Republic

songs JOIN artists

songs

artists

id	name	tempo	duration	artist_id	id	name	birthyear	label
1	Something Comforting	144	282	23	23	Porter Robinson	1992	Mom+Pop
2	Drive	142	196	45	45	Oh Wonder	1990	Republic

```
SELECT *  
FROM songs  
JOIN artists  
ON songs.artist_id = artists.id;
```

```
SELECT *  
FROM songs  
JOIN artists  
ON songs.artist_id = artists.id;
```

```
SELECT *  
FROM songs  
JOIN artists  
ON songs.artist_id = artists.id  
WHERE artists.name = "Oh Wonder";
```

6.sql

List the names of songs that are by Post Malone.

7.sql

Find the average energy of songs that are by Drake.

8.sql

List the names of the songs that feature other artists.

This is CS50

Week 7